LEVEL 15 — STACKS

We now move into the first data structure where **the order in which elements are removed matters**.

The central rule is:

> **LIFO — Last In, First Out**

Think of a stack of plates:

TOP
↓
[30]
[20]
[10]

If you remove one plate, `30` comes out first.

We'll implement stacks from scratch first, then explore how **C, Java, and Python provide different abstractions for the same concept**.

15A — Understand the Stack

Q118 — Basic Stack Operations

Design a stack supporting:

push(x)
pop()
peek()
isEmpty()
size()

For:

push(10)
push(20)
push(30)

the stack should be:

TOP

↓
30
20
10

Then:

pop()

should return:

30

Q119 — Stack Using an Array

Implement a stack using an array.

Start with:

capacity = 5

Support:

push()
pop()
peek()
isEmpty()
isFull()
size()

Test:

push(10)
push(20)
push(30)
pop()
peek()
size()

15B — Stack Boundary Conditions

Q120 — Empty Stack

Create an empty stack and call:

pop()
peek()

Decide what your implementation should do.

Compare the natural approaches:

C

Possible approaches:

return special value
return status
use output parameter

Java

Investigate what the standard stack/deque APIs do when removing from an empty structure.

Python

Investigate what happens when removing from an empty `list` or `deque`.

You should explicitly observe the exceptions/errors rather than guessing.

Q121 — Full Stack

For a fixed-size array stack:

capacity = 3

perform:

push(10)
push(20)
push(30)
push(40)

What happens?

Implement explicit overflow detection.

15C — Stack Implementation in Three Languages

Q122 — Build a Complete Stack

Implement the following from scratch.

C

Use:

struct Stack

and dynamically allocated memory where appropriate.

Explore:

malloc()
calloc()
realloc()
free()

from:

<stdlib.h>

Your stack should support:

create()
push()
pop()
peek()
isEmpty()
isFull()
size()
destroy()

Java

Create:

class Stack

with private state.

Implement:

push()
pop()
peek()
isEmpty()
size()

Then consider:

private
public
static
final

and why encapsulation matters.

Python

Create:

class Stack

using a Python `list`.

Implement:

push()
pop()
peek()
is_empty()
size()

Then create another version using:

collections.deque

Compare the two.

15D — Stack vs Standard Library

Q123 — Investigate Existing Implementations

Don't use your own implementation.

Find the idiomatic stack implementation in each language.

C

There isn't a built-in general-purpose stack type.

Investigate how you would build one from:

array
struct
dynamic memory

Java

Investigate:

java.util.Deque
java.util.ArrayDeque
java.util.Stack

Important question:

> Why is `Deque` / `ArrayDeque` generally preferred over the legacy `Stack` class?

Investigate methods such as:

push()
pop()
peek()
addFirst()
removeFirst()
peekFirst()

Understand the relationship between a **deque** and a **stack**.

Python

Investigate:

list
collections.deque

Compare:

stack.append(x)stack.pop()

with:

deque.append(x)deque.pop()

15E — Reverse Using a Stack

Q124

Given:

"HELLO"

reverse it using a stack.

Expected:

"OLLEH"

First implement manually.

Then solve using the language's natural stack abstraction.

15F — Balanced Parentheses

Q125 — Basic Parentheses

Determine whether:

()

is valid.

Test:

()
(())
()()
(())

(
)
(()
())

Use a stack.

Q126 — Multiple Bracket Types

Now support:

()
{}
[]

Examples:

"{[()]}" → valid
"{[(])}" → invalid
"([{}])" → valid
"[(])" → invalid

Create:

isBalanced(expression)

Q127 — Ignore Non-Bracket Characters

Input:

```
if (a[0] == '{')
    print("hello");
```

Your program should consider only:

()
[]
{}

and ignore everything else.

This is a good exercise in combining:

- strings
- character classification
- stack
- conditions

15G — Stack and Strings

Q128 — Remove Adjacent Duplicates

Given:

"abbaca"

remove adjacent duplicates repeatedly.

Expected:

"ca"

Reason:

abbaca
↓
aaca
↓
ca

Use a stack.

Test:

aaaa
aabbcc
abc
aabccbaa

15H — Undo Simulation

Q129 — Simple Undo

You have operations:

type A
type B
type C
undo
type D
undo

Represent the current text after every operation.

Use a stack to implement:

type(character)
undo()
current()

Q130 — Two-Stack Undo/Redo

Now maintain:

undoStack
redoStack

Operations:

type()
undo()
redo()

Test:

type(A)
type(B)
type(C)
undo()
undo()
redo()
type(D)

Important question:

> What should happen to the redo stack after a new operation following an undo?

This is a small but very useful example of **state management using data structures**.

15l — Stack-Based Number Problems

Q131 — Decimal to Binary

Convert:

to:

1010

using a stack.

Then test:

0
1
2
8
16
100

Q132 — Decimal to Any Base

Generalize:

decimalToBase(number, base)

Support:

base = 2
base = 8
base = 16

Investigate how you represent digits above 9.

For example:

10 → A
11 → B

15J — Postfix Expressions

This is our first real stack algorithm.

Q133 — Evaluate Postfix

Evaluate:

2 3 +

Result:

5

Then:

2 3 + 4 *

Result:

20

because:

2 + 3 = 5
5 × 4 = 20

Implement:

evaluatePostfix(expression)

Q134 — More Operators

Support:

+
-
*
/
%

Test:

5 2 -
10 3 %
8 2 /

Determine how integer division should behave in:

- C
- Java
- Python

This is an excellent language comparison.

For example, investigate the difference between:

-7 / 2

across the three languages.

15K — Infix → Postfix

Q135

Convert:

A + B

into:

A B +

Then:

A + B * C

into:

A B C * +

Then:

(A + B) * C

into:

A B + C *

You will need to understand:

- precedence
- associativity
- parentheses
- operators
- stack

Q136 — Operator Precedence

Before coding, determine the postfix form of:

A + B * C
(A + B) * C

A * B + C * D
A + B * C - D
(A + B) / (C - D)

Only after doing this manually should you implement the algorithm.

15L — Stack and Recursion

Q137 — Simulate Recursion

Take:

factorial(5)

and manually represent the calls:

factorial(5)
factorial(4)
factorial(3)
factorial(2)
factorial(1)

Then the returns:

1
2
6
24
120

Ask:

> What data structure is the runtime essentially using to remember these calls?

Then compare this with the stack data structure you've just implemented.

15M — Stack Complexity

Q138

For each operation:

push
pop
peek
isEmpty
search

determine its complexity for:

Array stack

Linked-list stack

Java `ArrayDeque`

Python `list`

Python `deque`

Don't just memorize `O(1)`.

Ask:

> What has to physically happen for this operation?

15N — Stack Using Linked List

Q139

Implement a stack using:

Node

instead of an array.

Example:

TOP
↓
30 → 20 → 10 → NULL

Implement:

push()
pop()

peek()
isEmpty()
size()

Q140 — Compare Two Implementations

Implement both:

ArrayStack
LinkedStack

Compare:

- memory usage
- overflow
- resizing
- insertion/removal
- cache locality
- complexity
- implementation complexity

Do this conceptually first.

LEVEL 16 — QUEUES

Now we switch from:

> **LIFO**

to:

> **FIFO — First In, First Out**

Think:

```
FRONT                REAR
↓                    ↓
10 → 20 → 30 → 40
```

`10` leaves first.

16A — Basic Queue

Q141 — Queue Operations

Implement:

enqueue(x)
dequeue()
front()
rear()
isEmpty()
size()

Test:

enqueue(10)
enqueue(20)
enqueue(30)

dequeue()

Expected:

10

16B — Array Queue

Q142

Create a fixed-size queue:

capacity = 5

Support:

enqueue()
dequeue()
front()
rear()
isEmpty()
isFull()

16C — The Important Queue Problem

Q143 — Wasted Space

Consider:

capacity = 5

Perform:

enqueue(10)
enqueue(20)
enqueue(30)
enqueue(40)
enqueue(50)

dequeue()
dequeue()

Now try:

enqueue(60)

A naïve array queue may claim that it is full even though there are two free positions.

Why?

Draw the array.

This motivates the circular queue.

16D — Circular Queue

Q144

Implement:

CircularQueue

using:

front
rear
size

capacity

Understand exactly what happens when:

rear

reaches the end of the array.

Q145 — Circular Queue Trace

For capacity 5, execute:

```
enqueue(10)
enqueue(20)
enqueue(30)
dequeue()
dequeue()
enqueue(40)
enqueue(50)
enqueue(60)
```

After every operation record:

```
array
front
rear
size
```

This should be done **before running the program**.

16E — Queue Implementations in Three Languages

Q146

Implement a queue from scratch in:

C

Use:

struct Queue

and dynamic memory where appropriate.

Java

Create:

class Queue

Then investigate:

Queue
Deque
ArrayDeque
LinkedList

from:

java.util

Python

Implement using:

list

first.

Then ask:

> Why is using `list.pop(0)` potentially undesirable for a queue?

Then investigate:

collections.deque

and use:

append()
popleft()

16F — Queue Library Investigation

Q147

Compare these APIs.

Java

ArrayDeque
LinkedList
PriorityQueue

Investigate:

add()
offer()
remove()
poll()
element()
peek()

Understand the difference between methods that:

- throw an exception
- return a special value

Python

Investigate:

collections.deque

Methods:

append()
appendleft()
pop()
popleft()
extend()
extendleft()
rotate()

Test the optional arguments and behavior.

16G — Queue Applications

Q148 — Printer Queue

Suppose print jobs arrive:

A
B
C
D

and the printer processes one job at a time.

Simulate:

addJob()
processJob()
nextJob()
pendingJobs()

Q149 — Customer Queue

Customers arrive:

A
B
C
D

Each customer has a service time.

Example:

A → 5 min
B → 2 min
C → 7 min
D → 3 min

Calculate:

- processing order
- waiting time
- completion time

Use a queue.

16H — Queue + Stack

Q150 — Reverse a Queue

Given:

10 → 20 → 30 → 40

produce:

40 → 30 → 20 → 10

using a stack.

This forces you to combine two data structures.

16I — Queue Using Two Stacks

Q151

Implement a queue using:

stack1
stack2

Support:

enqueue()
dequeue()

Example:

enqueue(1)
enqueue(2)
enqueue(3)

dequeue() → 1
dequeue() → 2

Understand why elements need to move between the two stacks.

16J — Stack Using Two Queues

Q152

Now reverse the idea.

Implement a stack using:

queue1
queue2

Support:

push()
pop()
peek()

Compare the complexity of the different implementation strategies.

LEVEL 17 — HASHING / MAPS / DICTIONARIES

Now we move to one of the most important practical data structures:

Key → Value

Examples:

"Samad" → 95
"Ali" → 87
"John" → 91

17A — Frequency Counting

Q153

Given:

[1,2,2,3,3,3,4,4,4,4]

produce:

1 → 1
2 → 2
3 → 3
4 → 4

First solve without a hash table.

Then solve using the language's map structure.

17B — Character Frequency

Q154

Given:

"programming"

produce frequency counts.

Then modify it to:

> Find the first character that occurs only once.

Example:

"swiss"

Expected:

w

17C — Word Frequency

Q155

Given:

"this is a test this is another test"

produce:

this → 2
is → 2
a → 1
test → 2
another → 1

Handle:

- uppercase
- punctuation
- multiple spaces

17D — Map APIs

Q156 — C

C has no built-in `HashMap`.

Design a basic key-value structure.

Start with:

struct Entry

containing:

key
value

Then explore how hashing would work.

Don't implement collision resolution yet.

Q157 — Java

Investigate:

java.util.HashMap

Methods:

put()
get()
remove()
containsKey()
containsValue()
getOrDefault()
putIfAbsent()
replace()

replaceAll()
compute()
computeIfAbsent()
computeIfPresent()
merge()
size()
isEmpty()
clear()
keySet()
values()
entrySet()

Don't memorize them.

Build small examples that force you to discover when each is useful.

Q158 — Python

Investigate:

dict

Methods:

get()
keys()
values()
items()
pop()
popitem()
setdefault()
update()
clear()
copy()
fromkeys()

Then investigate:

collections.defaultdict
collections.Counter

17E — Three-Language Frequency Challenge

Q159

Solve:

> Find the 10 most frequent words in a paragraph.

C

Build your own data structure.

Java

Use:

HashMap

then investigate whether the Java standard library gives you useful collection utilities for sorting the entries.

Python

Start with:

dict

then solve using:

Counter

Compare the amount of code required.

17F — First Non-Repeating Element

Q160

Given:

[4,5,1,2,1,4,5]

find the first element occurring exactly once.

Expected:

2

Solve:

1. Without hashing
2. With hashing

Compare complexity.

17G — Two Sum

Q161

Given:

[2,7,11,15]

and:

target = 9

find two elements whose sum is the target.

Expected:

2 + 7 = 9

First solve using nested loops.

Then solve using a hash map.

This is the first problem where you should clearly see:

> **A data structure can change an algorithm's complexity.**

17H — Duplicate Detection

Q162

Determine whether an array contains duplicates.

[1,2,3,4] → No
[1,2,3,1] → Yes

Solve:

1. Nested loops
2. Sorting
3. Hash set

Compare their complexity and memory requirements.

17I — Hash Set Investigation

Q163

Investigate sets.

Java

HashSet

Explore:

add()
remove()
contains()
size()
isEmpty()
clear()

Then compare:

HashSet
LinkedHashSet
TreeSet

Python

Investigate:

set
frozenset

and:

add()
remove()
discard()

pop()
union()
intersection()
difference()
symmetric_difference()
issubset()
issuperset()

C

Ask:

> Why doesn't the C standard library provide an equivalent built-in hash set?

This will lead naturally into implementing one yourself.

17J — Collision Concept

Q164

Suppose your hash function is:

hash(key) = key % 5

Insert:

10
15
20
7
12

Determine where each goes.

Then identify the collisions.

Q165 — Collision Strategies

Understand these two concepts:

Separate chaining

bucket 0 → 10 → 15 → 20
bucket 1 → ...

Open addressing

bucket 0 → occupied
bucket 1 → occupied
bucket 2 → next candidate

You don't need to implement either yet.

The goal is simply to understand:

> **Why does a hash table need collision handling?**

LEVEL 18 — TREES

Now we move into hierarchical data.

Start with the simplest possible tree.

Q166 — Create a Binary Tree

Create:

```
    10
   /  \
  5    20
 / \
3   7
```

Represent it in:

- C
- Java
- Python

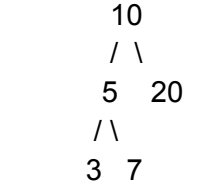
18A — Tree Traversal

Q167

Implement:

preorder()
inorder()
postorder()

For:



Determine the expected output for each.

18B — Level Order

Q168

Implement:

levelOrder()

Expected:

10 5 20 3 7

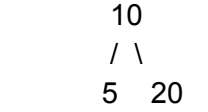
Notice what data structure naturally helps here.

Queue.

18C — Tree Height

Q169

Find the height of:



/ \
3 7

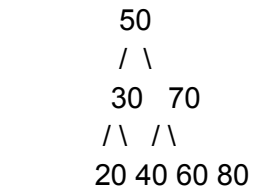
Then test:

single node
empty tree
completely skewed tree
balanced tree

LEVEL 19 — BINARY SEARCH TREE

Q170 — BST Search

Create:



Search for:

20
60
90

Q171 — BST Insertion

Implement:

insert(root, value)

Insert:

50
30
70
20
40
60
80

Then print inorder.

What do you notice?

> **Inorder traversal of a BST produces sorted order.**

Q172 — BST Minimum and Maximum

Find:

minimum()
maximum()

without traversing the entire tree unnecessarily.

Q173 — BST Deletion

Implement deletion for:

1. Leaf node
2. Node with one child
3. Node with two children

Test:

delete(20)
delete(30)
delete(50)

This is the first tree problem where you should slow down and draw the tree after every operation.

LEVEL 20 — HEAPS / PRIORITY QUEUES

We will introduce:

- min heap

- max heap
- insert
- extract
- peek
- heapify
- priority queues

Then compare:

C

Manual heap implementation.

Java

PriorityQueue

Python

heapq

LEVEL 21 — BASIC GRAPHS

Finally, the first graph foundation:

Q174 — Represent a Graph

Represent:

```
A --- B
|     |
C --- D
```

using:

1. Adjacency matrix
2. Adjacency list

Implement in all three languages.

Q175 — BFS

Starting from `A`, perform BFS.

Q176 — DFS

Starting from `A`, perform DFS.

Implement:

1. Recursive DFS
2. Stack-based DFS

Then compare BFS and DFS.

The progression from here

So the complete data-structure ladder is now:

LEVEL 11 Searching
↓
LEVEL 12 Sorting
↓
LEVEL 13 Recursion
↓
LEVEL 14 Linked Lists
↓
LEVEL 15 Stacks
↓
LEVEL 16 Queues
↓
LEVEL 17 Hashing / Maps / Sets
↓
LEVEL 18 Binary Trees
↓
LEVEL 19 Binary Search Trees
↓
LEVEL 20 Heaps / Priority Queues
↓
LEVEL 21 Basic Graphs

After that, I would **not immediately jump into advanced algorithms**. We should first do a **language-specific consolidation round** where the same data structures are revisited through the lens of each language:

C: pointers → structures → dynamic memory → callbacks → function pointers → generic-looking data structures using `void *`

Java: interfaces → generics → `Comparable` → `Comparator` → collections framework → iterators → OOD implementation of the structures

Python: duck typing → protocols → iterators → generators → special methods → `collections` → custom container classes

Only after that would I start the **OS + multithreading + networking** track.